

PRACTICAL-3

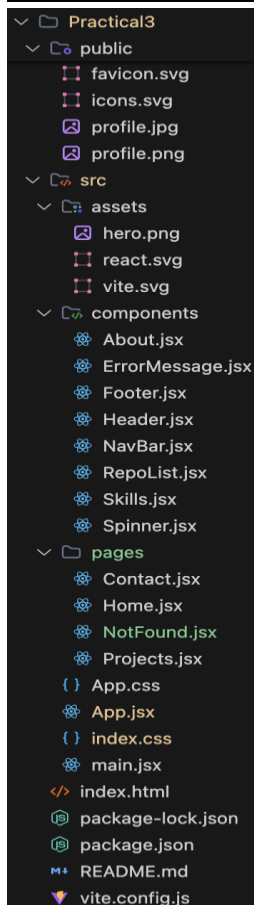
PROBLEM DEFINITION:

- Integrate a public REST API (e.g., GitHub API) into the portfolio application to fetch and display a list of repositories dynamically.
- Use the Projects page built in Practical 2 as the integration point.
- Implement a loading spinner component shown while the request is in progress.
- Implement an error message component shown if the API call fails.
- Render at least the repository name and URL for each item returned.

SUPPLEMENTARY PROBLEMS:

- Add a retry button that re-triggers the fetch when an error occurs.
- Add a search input that filters the rendered repository list by name.
- Display the repository's star count alongside its name.

ARCHITECTURE / DIAGRAM:



OUTPUT (SUPPLEMENTARY PROBLEMS INCLUDED):

Jainam Kamani

Home Projects Contact

GitHub Repositories

Fetch Repos Test Error & Retry Button

Search repositories by name...

Showing 8 of 8 repositories

boysenberry-repo-1

Testing

26 forks

View Repository

git-consortium

This repo is for demonstration purposes only.

178 forks

View Repository

hello-world

My first repository on GitHub.

294 forks

View Repository

Hello-World

My first repository on GitHub!

6484 forks

View Repository

linguist

Language Savant. If your repository's language is being reported incorrectly, send us a pull request!

Ruby 278 forks

View Repository

octocat.github.io

No description provided for this repository.

CSS 564 forks

View Repository

Spoon-Knife

This repo is for demonstration purposes only.

HTML 158637 forks

View Repository

test-repo1

No description provided for this repository.

46 forks

View Repository

Let's Connect

Feel free to reach out for collaborations or project inquiries.

jainamkamani95@gmail.com [GitHub](#) [LinkedIn](#)

© 2026 Jainam Kamani. All rights reserved.
Junagadh, Gujarat, India

github.com

Jainam Kamani | Student Portfolio

GitHub - octocat/boysenberry-repo-1: Testing - GitHub

Platform Solutions Resources Open Source Enterprise Pricing

Search Sign in Sign up

octocat / boysenberry-repo-1

forked from violet-org/boysenberry-repo

Notifications Fork 26 Star 471

Code Pull requests 1 Actions Projects Wiki Security and quality Insights

master 8 Branches 0 Tags

Go to file Code

This branch is up to date with violet-org/boysenberry-repo:master.

jmarlena

Set theme: "jekyll-theme-minimal" in _config.yml.

d09e445 · 10 years ago

4 Commits

README.md

Create Jekyll site.

10 years ago

READTHIS.md

add new version of readme

10 years ago

_config.yml

Create Jekyll site.

10 years ago

README

Welcome to GitHub Pages

You can use the [editor on GitHub](#) to maintain and preview the content for your website in Markdown files.

Whenever you commit to this repository, GitHub Pages will run [Jekyll](#) to rebuild the pages in your site, from the content in your Markdown files.

Markdown

About

Testing

Readme

Activity

471 stars

18 watching

26 forks

Report repository

Releases

No releases published

Contributors

No contributors

Jainam Kamani

Home Projects Contact

API Request Failed

Failed to fetch repositories for 'INVALID_TEST_USER' (Status: 404)

Retry Fetch

Let's Connect

Feel free to reach out for collaborations or project inquiries.

jainamkamani95@gmail.com [GitHub](#) [LinkedIn](#)

© 2026 Jainam Kamani. All rights reserved.
Junagadh, Gujarat, India

KEY QUESTIONS / ANALYSIS:**1) Why is useEffect required to trigger a fetch on component mount instead of calling fetch directly in the component body?**

useEffect is used because fetching data is a side effect. If fetch() is called directly inside the component body, it can execute every time the component re-renders, causing unnecessary API requests and potentially creating an infinite request loop. With useEffect(() => {...}, []), the fetch operation runs once when the component mounts.

2) What is the difference between a loading state and an error state, and why must both be handled separately?

The loading state indicates that the API request is currently in progress and the application is waiting for a response. The error state indicates that the request has failed or the API returned an error. They must be handled separately because they represent different situations: loading requires showing a spinner or progress indicator, while an error requires showing an appropriate error message to the user.

3) How would the user experience change if loading and error states were not implemented?

Without a loading state, users may think the application is frozen or not responding while the API request is processing. Without an error state, users may see an empty Projects page or unclear content when the API fails, without knowing what went wrong. Implementing both states provides clear feedback, better usability, and a more reliable user experience.

GITHUB REPOSITORY:

<https://github.com/9081495207/ITUE301-Advanced-Web-Development-Frameworks>